



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 10

NOMBRE COMPLETO: Pablo Arturo Gonzalez Cuellar

N° de Cuenta: 319241013

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 10/11/2025

CALIFICACIÓN: _____

1. Animar algún objeto por medio de Keyframes (que no sea ninguno de los personajes principales ni humanoide) requiere compartir capturas de pantalla donde se vea la impresión de los keyframes guardados o reproducidos en consola y guardados en un archivo de tipo txt y compartir los keyframes para que yo pueda reproducir su animación. La única restricción es que la Animación debe de representar el movimiento de un cuerpo en la vida real lógico, por ejemplo: una caja que va rodando no es un movimiento lógico. Un helado flotando no es un movimiento lógico.

Debe de presentarse al objeto animado ya integrado en su proyecto final

Para el desarrollo de esta práctica se quiso innovar en la parte de animación por keyframes.

Primero, fue necesario definir, como ya se había implementado, una estructura **FRAME** que pudiera almacenar el estado de nuestro objeto en un punto específico en el tiempo

```
// Estructura para almacenar
struct FRAME
{
    float posX;
    float posY;
    float posZ;
    float rotY;
};
```

Después creamos la función principal de nuestra animación, la cual se mandará a llamar en cada fotograma que guardemos, lo que hace es revisar en qué *FrameIndex* (guarda el cuadro clave) vamos al avanzar y usa la interpolación para calcular la posición exacta :

```
// Posición y Rotación actual del Globo
float globoPosX = 0.0f;
float globoPosY = 50.5f;
float globoPosZ = -180.0f;
float globoRotY = 90.0f;
```

Entre el cuadro al que se está accediendo y el siguiente

```

// Función principal de Animación
void animate(float& posX, float& posY, float& posZ, float& rotY,
std::vector<FRAME>& keyframes, int& frameIndex, float& playIndex, bool& play) {

    if (play && keyframes.size() > 1) {

        // Cuando llega a 1.0, pasamos al siguiente frame.
        if (playIndex > 1.0f) {
            playIndex = 0.0f;
            frameIndex++;
        }

        // Si llegamos al final de la animación, la detenemos y reiniciamos.
        if (frameIndex >= keyframes.size() - 1) {
            frameIndex = 0;
            play = false;
            printf("Animacion terminada.\n");
        }

        // Frames actual y siguiente
        int currentFrame = frameIndex;
        int nextFrame = frameIndex + 1;

        // Interpolamos la posición y rotación entre el frame actual y el siguiente
        posX = interpolation(keyframes[currentFrame].posX, keyframes[nextFrame].posX, playIndex);
        posY = interpolation(keyframes[currentFrame].posY, keyframes[nextFrame].posY, playIndex);
    }
}

```

Y con esto ya tendríamos el control de los keyframes del globo.

Posteriormente, se creó nuestra lógica de movimiento del globo usando las teclas “F,T,G y H” y las teclas *UP* y *DOWN* del teclado para posicionar el globo libremente por el escenario.

```

float globoMoveSpeed = 20.0f;
if (mainWindow.getKeys()[GLFW_KEY_T]) {
    globoPosZ -= globoMoveSpeed * deltaTime;
}
if (mainWindow.getKeys()[GLFW_KEY_B]) {
    globoPosZ += globoMoveSpeed * deltaTime;
}
if (mainWindow.getKeys()[GLFW_KEY_F]) {
    globoPosX -= globoMoveSpeed * deltaTime;
}
if (mainWindow.getKeys()[GLFW_KEY_H]) {
    globoPosX += globoMoveSpeed * deltaTime;
}
if (mainWindow.getKeys()[GLFW_KEY_UP]) {
    globoPosY += globoMoveSpeed * deltaTime;
}
if (mainWindow.getKeys()[GLFW_KEY_DOWN]) {
    globoPosY -= globoMoveSpeed * deltaTime;
}

```

Después, se implementó la lógica para el grabado de la posición del globo como un nuevo keyframe con la tecla “L”, en este bloque se toma la lectura de la posición del globo en X,Y y Z y la rotación de éste, añade ese frame a la lista de animación *KeyFrameGlobo* y muestra el mensaje en la consola si es que se guardó exitosamente el keyframe

```
// Lógica de Keyframes del Globo (Teclas 'L', 'K', 'G', 'C')
if (mainWindow.getKeys()[GLFW_KEY_L] && !teclaPresionada) {
    teclaPresionada = true;
    FRAME newFrame;
    newFrame.posX = globoPosX;
    newFrame.posY = globoPosY;
    newFrame.posZ = globoPosZ;
    newFrame.rotY = globoRotY;
    KeyFrameGlobo.push_back(newFrame);
    printf("Keyframe %d guardado: (%.2f, %.2f, %.2f)\n", (int)KeyFrameGlobo.size(), newFrame.posX,
}
if (!mainWindow.getKeys()[GLFW_KEY_L]) {
    teclaPresionada = false;
}
```

Después, se hizo la asignación de la tecla “K” para iniciar la animación del globo. Esta animación activa la bandera *playGlobo=true* la cual al estar en este estado, sella función *animate* empieza a actualizar las variables de traslación del globo usando interpolación

```
if (mainWindow.getKeys()[GLFW_KEY_K] && !teclaKPresionada) {
    teclaKPresionada = true;
    if (KeyFrameGlobo.size() > 1) {
        playGlobo = true;
        FrameIndexGlobo = 0;
        playIndexGlobo = 0.0f;
        printf("Reproduciendo animación del globo...\n");
    }
    else {
        printf("¡Error! Necesitas al menos 2 keyframes para reproducir.\n");
    }
}
```

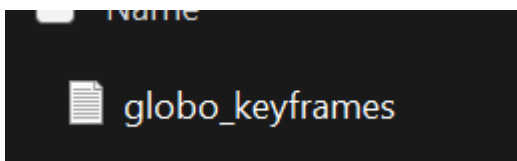
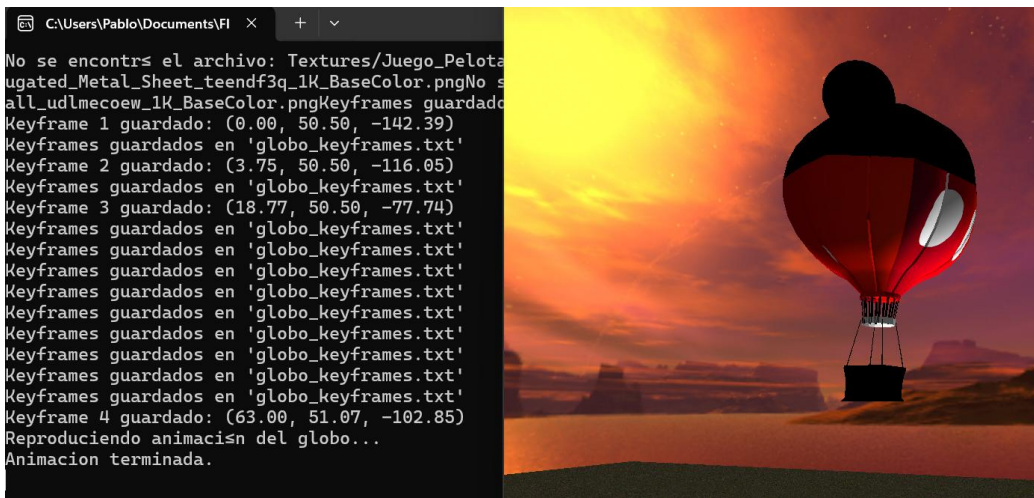
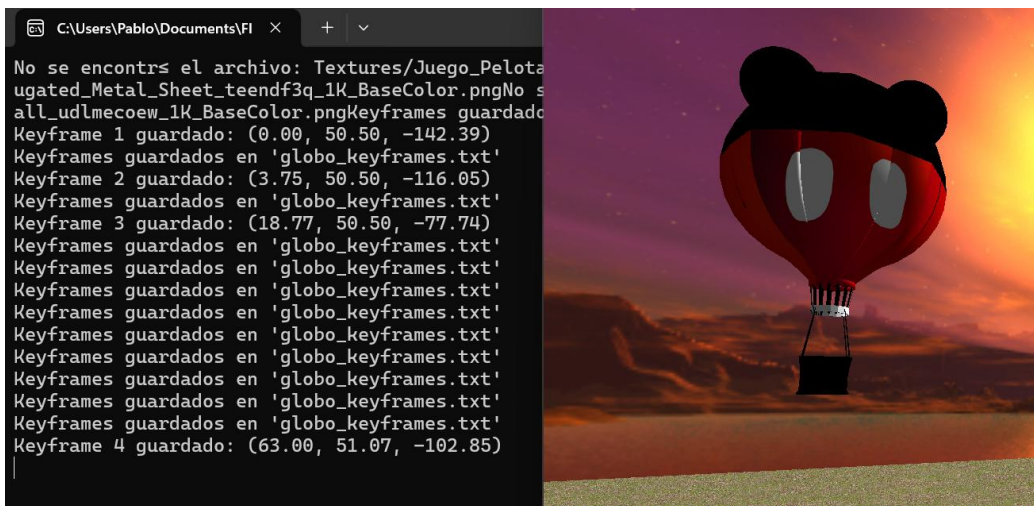
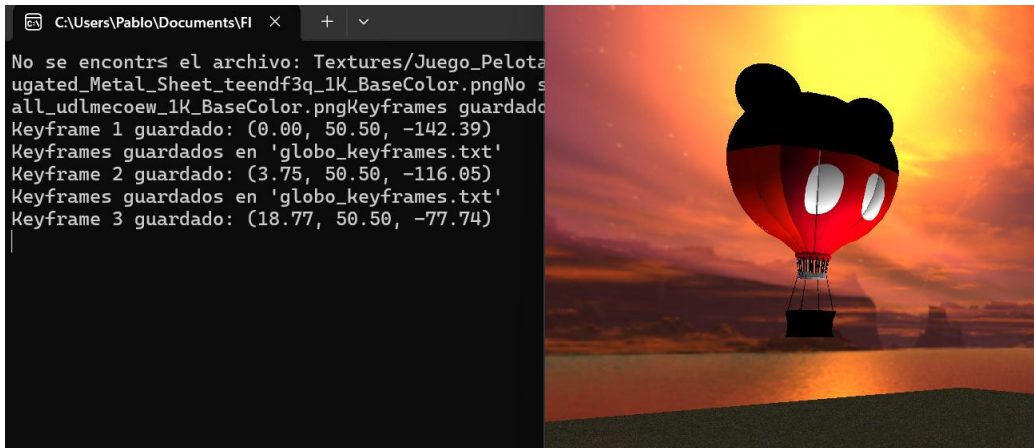
Finalmente, agregamos la ultima lógica con la tecla “G” para guardar el mapeado de los keyframes generados en un archivo llamado “*globo_keyframes.txt*”

```
if (mainWindow.getKeys()[GLFW_KEY_G] && !teclaGPresionada) {
    teclaGPresionada = true;
    saveKeyframes(KeyFrameGlobo, "globo_keyframes.txt");
}
if (!mainWindow.getKeys()[GLFW_KEY_G]) {
    teclaGPresionada = false;
}
```

Renderizamos el globo y le pasamos las variables de traslación y rotación

```
// Globo
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(globoPosX, globoPosY, globoPosZ));
model = glm::rotate(model, glm::radians(globoRotY), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(14.0f, 14.0f, 14.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
Globo_M.RenderModel();
```

CAPTURAS DE EJECUCIÓN:



PROBLEMAS SURGIDOS:

Afortunadamente, en esta práctica no se tuvo ningún problema

CONCLUSIÓN

En esta práctica se reforzó la construcción de una animación compleja en OpenGL. A diferencia de las animaciones con traslaciones senoidales y cosenoidales, el sistema de animación por keyframes nos da un control total sobre la ruta, el tiempo y el comportamiento de un objeto. Implementar el sistema de grabación, guardado y reproducción permitió crear una animación personalizada para el globo dándonos un plus y adelanto para poder implementarlo en el proyecto final de la materia